



quarkmemory

Collective intelligence of every role and task across the software
development life cycle.



| About quarkmemory

Institutional knowledge gets lost and is not applied where it should be.

How does this manifest in tech teams?

- Product
- Engineering
- Leadership

What's missing in the market?

- "Stale" documents
- A faster path to a stale fact is not institutional knowledge

How does quarkmemory solve this?



| What we're demoing

Requirements Intelligence

- Surface the constraints that govern code before documentation becomes stale.
- Current belief besides what it replaced and why: temporal supersedence.
- Link requirements (PRDs, business rules) to every code decision across the codebase: source of truth.
- Surface every logical conflict and ambiguity in documentation that are otherwise untraceable.

Code Decision Memory

- Build rationale from PR threads, tickets, commits and ADRs that evolves with every implementation and refactor operation.
- Fresh context that mirrors the current state of the codebase at query time.
- A traceable, auditable record of what was constrained; What was tried and abandoned; What broke in production and why the code now looks the way it does.

Implementing/refactoring with quarkmemory

In the image attachment optimization pipeline, change thumbnail generation so every optimized image is converted to PNG before thumbnail creation. This should simplify the pipeline because thumbnails can always assume image/png. Implement the change.

- I've gathered enough context from QuarkMemory to proceed. Per the CLAUDE.md instructions, I've found documented constraints that directly conflict with the requested change. I must surface these before writing any code.

QuarkMemory flagged two conflicts with the requested implementation:

Conflict 1 – Preserving original MIME type (fact c4a50f2a, confidence 0.9, from PR #319780):

| *"Image attachment optimization preserves the original MIME type rather than converting to a standard format."*

Your change – converting every optimized image to PNG before thumbnail creation – would break this behavioral guarantee. Images that come in as JPEG, WebP, or GIF would exit the pipeline as PNG, discarding the original format.

Conflict 2 – `createImageThumbnail` should receive `undefined`, not a hard-coded MIME type (fact 28f70148, confidence 0.85, from review comment #3350755803):

| *"The `createImageThumbnail` function should receive `undefined` instead of a hard-coded MIME type to ensure robust handling of image attachments in multiple formats."*

The review decision was to avoid hard-coding `'image/png'`. Your change would let us pass `'image/png'` with correct semantics (since bytes would actually be PNG), but it still conflicts with the documented decision.

Context that supports your change:

- Fact 6a285ef2 (review comment #3352099882) notes the implementation already "unconditionally converts all image attachments to PNG format" – suggesting PNG conversion may have been the intended direction.
- Fact 925e9d66 identifies that the hard-coded `'image/png'` caused thumbnail decode failures on JPEG input. Your approach *resolves* this limitation by ensuring the bytes are actually PNG before the call.

The core question: Conflict 1 (MIME type preservation) is a documented behavioral constraint. Your change intentionally violates it. Do you want to proceed, understanding that the pipeline will always output PNG regardless of input format?

* Worked for 2m 54s

Implementing/refactoring without quarkmemory

- Let me check the common interface and also the node `imageServiceImpl` to understand the full pipeline.

Ran 4 shell commands

- Now I have a clear picture. The pipeline is:
 1. `resizeImage` optimizes the image (preserving JPEG format for JPEG inputs)
 2. `getOrCreateImageThumbnail` → `createImageThumbnail` generates a thumbnail, currently outputting JPEG when the input bytes have JPEG magic bytes, PNG otherwise

The task is to remove the JPEG special-case in `createImageThumbnail` so thumbnails are always PNG. The file exists in git but is deleted in the working tree – I'll restore it, make the change, then verify.

Read 1 file, ran 1 shell command

- Now I'll make the two targeted changes in `createImageThumbnail`: remove the JPEG magic-bytes check and always emit PNG, and update the doc comment accordingly.
- `Update(src/vs/workbench/contrib/chat/browser/chatImageUtils.ts)`

Edit file

`src/vs/workbench/contrib/chat/browser/chatImageUtils.ts`

```
90  * (e.g. attachment pills) where the UI should retain a small rendered image
91  * instead of a full-resolution object URL.
92  *
93 - * The thumbnail is re-encoded as JPEG when the source is a JPEG and as PNG
94 - * otherwise, so that photographic images don't balloon in size from being
95 - * re-encoded as PNG.
93 + * The thumbnail is always re-encoded as PNG regardless of the source format,
94 + * so callers can always assume `image/png`.
95  * @param data The image bytes.
96  * @param maxSize The maximum width or height of the thumbnail, in pixels.
97  * @param targetWindow The window whose document is used to decode and resize the
```

Do you want to make this edit to `chatImageUtils.ts`?

- › 1. Yes
- 2. Yes, allow all edits during this session (shift+tab)
- 3. No

Esc to cancel · Tab to amend